

# tf.compat.v1.scatter\_nd\_update Stay organized with collections Save and categorize content based on your preferences.

[https://www.tensorflow.org/api\\_docs/python/tf/compat/v1/scatter\\_nd\\_update](https://www.tensorflow.org/api_docs/python/tf/compat/v1/scatter_nd_update)

*Applies sparse updates to individual values or slices in a Variable.*

## Function Signatures

```
tf.compat.v1.scatter_nd_update( ref, indices, updates,  
                                use_locking=True, name=None )
```

## Code Examples

```
tf.compat.v1.scatter_nd_update(  
    ref, indices, updates, use_locking=True, name=None  
)
```

```
tf.compat.v1.scatter_nd_update(  
    ref, indices, updates, use_locking=True, name=None  
)
```

```
[d_0, ..., d_{Q-2}, ref.shape[K], ..., ref.shape[P-1]].
```

```
[d_0, ..., d_{Q-2}, ref.shape[K], ..., ref.shape[P-1]].
```

```
ref = tf.Variable([1, 2, 3, 4, 5, 6, 7, 8])
indices = tf.constant([[4], [3], [1], [7]])
updates = tf.constant([9, 10, 11, 12])
update = tf.compat.v1.scatter_nd_update(ref, indices, updates)
with tf.compat.v1.Session() as sess:
    print sess.run(update)
```

```
ref = tf.Variable([1, 2, 3, 4, 5, 6, 7, 8])
indices = tf.constant([[4], [3], [1], [7]])
updates = tf.constant([9, 10, 11, 12])
update = tf.compat.v1.scatter_nd_update(ref, indices, updates)
with tf.compat.v1.Session() as sess:
    print sess.run(update)
```

```
[1, 11, 3, 10, 9, 6, 7, 12]
```

```
[1, 11, 3, 10, 9, 6, 7, 12]
```

# Documentation

---

Applies sparse updates to individual values or slices in a Variable.

`ref` is a Tensor with rank  $P$  and `indices` is a Tensor of rank  $Q$ .

`indices` must be integer tensor, containing indices into `ref`.

It must be shape  $[d_0, \dots, d_{\{Q-2\}}, K]$  where  $0 < K \leq P$ .

The innermost dimension of `indices` (with length  $K$ ) corresponds to indices into elements (if  $K = P$ ) or slices (if  $K < P$ ) along the  $K$ th dimension of `ref`.

`updates` is Tensor of rank  $Q-1+P-K$  with shape:

For example, say we want to update 4 scattered elements to a rank-1 tensor to 8 elements. In Python, that update would look like this:

The resulting update to `ref` would look like this:

See `tf.scatter_nd` for more details about how to make updates to slices.

**## Returns**